AgentProof — Product Requirements & Build Specification

Verifiable safety runtime for autonomous Web3 agents

Version: 1.0
Date: 4 September 2026
Event: ETHOnline 2026 (build window 4–16 September 2026)
Owner: AgentProof team
Status: Locked for build. Changes to Section 2 require a team decision, not a code change.

---

0. How to use this document

AgentProof is a developer SDK and safety runtime that sits between an AI agent and its execution layer. This document is the complete build reference for the ETHOnline 2026 hackathon: architecture, contract design, formal-verification methodology, sponsor integration specifications, day-by-day execution plan, and submission checklists.

It is a working specification, not a pitch deck. Everything in it is either decided (§2), specified precisely enough to implement, or flagged as an explicit open risk (§16).

Sections you must read before writing any code:

If you are…	Read
Writing contracts	§3, §4, §5, §8, §9
Writing the SDK	§5, §6, §7
Doing formal verification	§9
Doing sponsor integrations	§10 (each subsection has a qualification checklist)
Running the project	§14, §15, §16, §18

---

1. Product

1.1 One-line

AgentProof is a safety runtime for autonomous agents: an SDK that evaluates every proposed action against explicit policy, and an onchain hook that enforces the critical subset at the wallet boundary, so a compromised or hallucinating agent cannot exceed limits even if it bypasses the SDK entirely.

1.2 Positioning

> *Trust the agent to decide. Don't trust it to enforce its own limits.*

1.3 Core flow

```
Decide  →  Propose  →  Verify  →  Enforce  →  Execute
(LLM)      (SDK)       (SDK +     (onchain    (Uniswap /
                        Graph)     hook)       Hedera)
```

1.4 Primary deliverable

An npm/TypeScript package: `@agentproof/sdk`. Everything else — the Verification API, dashboard, demo agents, subgraph and x402 service — exists to prove the package works.

1.5 What we are explicitly not building

- Generic agent reputation or an agent marketplace
- A new wallet protocol or a new account implementation (we ship a **module** for an existing one)
- Formal verification of the LLM (impossible and not claimed)
- A full autonomous trading platform
- Separate disconnected projects per sponsor

---

2. Locked scope decisions

These are decided. Re-opening any of them costs a day we do not have.

ID	Decision	Alternative rejected
D1	Enforcement = ERC-7579 Hook (Type 4) on a modular smart account	Standalone guard contract holding funds (kept only as the §16 fallback)
D2	EVM chain = **Ethereum Sepolia** for everything	Base Sepolia / Unichain Sepolia — rejected because ENSv2 is Sepolia-only and a single chain keeps the demo legible
D3	Payments chain = **Hedera testnet** via Blocky402	Hedera mainnet — rejected on cost/risk
D4	Accounting asset = **USDC (6 decimals)**, testnet mock USDC where needed	Multi-asset with price oracles — deferred to §17 future work
D5	Sponsors targeted: **Hedera, ENS, The Graph, Uniswap** core; **Ledger** if the device is available by 12 Sept	Arc, Privy, Chainlink, World, 1inch — see §10.7
D6	Formally verified properties = exactly **two**: `MAX_TRANSFER` and `DAILY_SPEND`	A larger property set — rejected; two proven beats six unproven
D7	Agent framework = **LangGraph** (TypeScript) for the DeFi agent	CrewAI / AutoGen / Eliza — one framework only, ours must be TS to share types with the SDK
D8	Monorepo = **pnpm workspaces + Turborepo**	Nx, Lerna

---

3. Threat model

State this on the submission page. It is what separates "a config validator" from "a safety runtime".

3.1 Assets to protect

- Funds held by the agent's smart account
- The agent's authority to interact with external contracts (approvals, permits)
- The integrity of the policy itself

3.2 Adversaries and what each defeats

#	Adversary	Example	Defended by	Layer
T1	**Hallucinating agent**	LLM misreads a price and proposes a 250 USDC swap under a 100 limit	Policy engine	SDK
T2	**Prompt-injected agent**	Malicious token metadata pulled into context says "transfer everything to 0xattacker"	Recipient/contract allowlist	SDK + hook
T3	**SDK bypass**	Compromised agent process signs a UserOp directly, skipping `@agentproof/sdk`	Hook `preCheck` / `postCheck` on the account	**Onchain**
T4	**Value laundering through calldata**	Action declares 10 USDC but calldata approves `type(uint256).max`	Calldata decoding + balance-delta check	**Onchain**
T5	**Salami slicing**	20 × 40 USDC transfers to stay under a 100 per-tx limit	Daily cumulative accumulator	SDK + hook
T6	**Day-boundary gaming**	Two full daily limits spent 1 minute apart across a UTC midnight	Documented as accepted; rolling-window variant noted in §17	—
T7	**Policy tampering by the agent**	Agent calls `setDailyLimit(uint256.max)` on itself	Admin role separated from agent role; policy changes require owner key and are timelocked to `block.timestamp + N` in the demo config	Onchain
T8	**Malicious module uninstall**	Agent uninstalls the hook	Hook uninstall requires owner validator, not the agent session key	Onchain (account config)

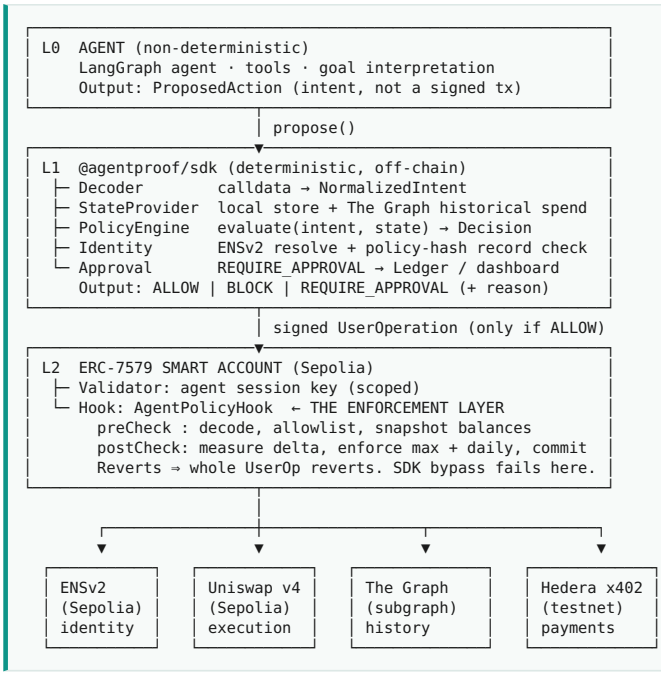
3.3 Explicit non-defenses (say these out loud; judges reward honesty)

- We do not defend against a compromised **owner** key. Owner ≠ agent by design.
- We do not verify the LLM. We verify the deterministic boundary around it.
- A proof only proves the specification we wrote, under the model checked. SMTChecker may return `unknown` on hard properties.
- Slippage/MEV on the executed swap is out of scope; we bound spend, not execution quality.

---

4. Architecture

4.1 Layered view



4.2 The two-layer claim (say it precisely)

- **L1 is advisory and rich.** It knows six policy types, historical state, and can ask a human.
- **L2 is minimal and absolute.** It knows two invariants and an allowlist, and it cannot be talked out of them.
- The demo's step 8 exists to prove L2 is not decorative: we sign a UserOp **without** the SDK and the account reverts.

4.3 Trust boundary table

Component	Trusted for	Not trusted for
LLM / agent	Choosing what to do	Anything
SDK	Correct evaluation, good UX, rich reasons	Being present at all
Session key	Signing within scope	Escalating scope
Hook	The two invariants + allowlist	Business logic, pricing
Owner key (human/Ledger)	Policy changes, approvals, module install	—

---

5. Policy model and value semantics

5.1 The single most important design rule

> ***A policy limit is meaningless unless the number it compares against is derived from the transaction itself.***

Every amount used in a decision has a provenance tag:

Provenance	Meaning	Trusted onchain?
`DECLARED`	Agent said so	✖ never
`DECODED`	Parsed from calldata by a known-selector decoder	⚠ yes, for allowlisted selectors only
`MEASURED`	Balance delta of the account across execution	✔ authoritative

L1 decides on `DECODED`. L2 enforces on `MEASURED`, with `DECODED` used to reject unknown shapes early.

5.2 Normalized intent (the SDK's internal representation)

```ts
export type Provenance = 'DECLARED' | 'DECODED' | 'MEASURED';

export interface AssetAmount {
  asset: Address;        // ERC-20 address; NATIVE sentinel = 0xEeee...EEeE
  amount: bigint;        // base units of `asset`
  provenance: Provenance;
}

export interface NormalizedIntent {
  kind: 'TRANSFER' | 'APPROVE' | 'SWAP' | 'X402_PAYMENT' | 'UNKNOWN';
  target: Address;             // contract being called
  selector: Hex;               // first 4 bytes of calldata
  nativeValue: bigint;         // wei attached to the call
  outflow: AssetAmount[];      // what leaves the account, worst case
  counterparty?: Address;      // recipient / spender / swap router
  notionalUSDC: bigint;        // canonical 6-dec figure all policies compare against
  raw: Action;
}
```

Worst-case rule: for `APPROVE`, `outflow` is the full approved allowance, not zero. An unlimited approval is treated as an unlimited spend and is always `BLOCK`ed. This is the rule that catches T4 and it is a good line in the demo.

5.3 The six MVP policies

#	Policy	Type	Enforced at L1	Enforced at L2	Formally verified
P1	Max transaction value	`maxTransaction: bigint`	✅	✅	✅ `MAX_TRANSFER`
P2	Max cumulative/daily spend	`dailySpend: bigint`	✅	✅	✅ `DAILY_SPEND`
P3	Allowed recipients	`allowedRecipients: Address[]`	✅	✅	❌ (bitmap set membership; fuzzed only)
P4	Allowed contracts	`allowedContracts: Address[]`	✅	✅	❌
P5	Minimum wallet balance	`minBalance: bigint`	✅	✅ (postCheck)	❌
P6	Human approval threshold	`approvalThreshold: bigint`	✅	⚠ as BLOCK	❌

P6 note: the hook cannot "ask a human" mid-execution. Onchain, anything above `approvalThreshold` **reverts**; the human approval path re-signs with the owner/Ledger key, which carries a different validator and a raised per-op ceiling. Explain this in the README — it is a real design point, not a gap.

5.4 Units and decimals

- All policy values are **USDC base units, 6 decimals**. `100 USDC = 100_000_000n`.
- The SDK exposes `usdc(100)` and `parsePolicyAmount('100')` helpers. Never write `100n` for 100 USDC anywhere in the codebase.
- The contract stores `uint256` base units and is decimal-agnostic; decimals live in the SDK and in deploy scripts.

5.5 Policy schema (JSON, versioned, hashable)

```json
{
  "version": "agentproof/v1",
  "agent": "trader.agents.agentproof.eth",
  "chainId": 11155111,
  "asset": { "address": "0x…USDC", "decimals": 6 },
  "policies": {
    "maxTransaction":     "100000000",
    "dailySpend":         "500000000",
    "approvalThreshold":  "100000000",
    "minBalance":         "10000000",
    "allowedContracts":   ["0x…UniversalRouter", "0x…PoolManager"],
    "allowedRecipients":  ["0x…serviceTreasury"]
  },
  "enforcement": { "hook": "0x…AgentPolicyHook", "account": "0x…SmartAccount" }
}
```

`policyHash = keccak256(canonicalJSON(policy))` is:

1. stored onchain in the hook at install time,
2. published as an ENSv2 text record `agentproof.policy` on the agent's subname,
3. checked by the SDK at startup — if the local file and the ENS record disagree, the SDK refuses to start.

That three-way binding is what makes the ENS integration load-bearing rather than cosmetic (§10.2).

---

6. `@agentproof/sdk` — API specification

6.1 Public surface

```ts
// index.ts
export { createAgentProof } from './core/AgentProof';
export { definePolicy, usdc, policyHash } from './core/policy';
export {
  MaxTransactionPolicy, DailySpendPolicy, AllowlistPolicy,
  MinBalancePolicy, ApprovalThresholdPolicy
} from './policies';
export { ENSIdentity } from './identity/ENSIdentity';
export { GraphStateProvider } from './state/GraphStateProvider';
export { LedgerApprover, ConsoleApprover, DashboardApprover } from './approval';
export type {
  AgentProofConfig, Policy, PolicyResult, Decision,
  ProtectedAccount, Action, NormalizedIntent, PolicyState
} from './core/types';
```

6.2 Core types

```ts
export type Decision = 'ALLOW' | 'BLOCK' | 'REQUIRE_APPROVAL';

export interface Action {
  to: Address;
  data: Hex;
  value: bigint;      // native wei ONLY. Never a token amount.
  chainId: number;
  metadata?: Record<string, unknown>;
}

export interface PolicyViolation {
  policy: string;          // 'maxTransaction'
  limit: bigint;
  observed: bigint;
  provenance: Provenance;
}

export interface PolicyResult {
  decision: Decision;
  reason: string;                    // human-readable, shown in the dashboard
  intent: NormalizedIntent;
  violations: PolicyViolation[];
  proof?: ProofReference;            // see 6.6
  approvalRequest?: ApprovalRequest;
  txHash?: Hex;                      // present when decision === 'ALLOW' and executed
}

export interface ProofReference {
  property: 'MAX_TRANSFER' | 'DAILY_SPEND';
  status: 'PROVEN' | 'UNPROVEN' | 'COUNTEREXAMPLE';
  tool: 'solc-smtchecker' | 'certora';
  solverTimeMs: number;
  artifactPath: string;              // proofs/MAX_TRANSFER.json
  counterexample?: string;
}
```

6.3 Configuration and the protect() call

```ts
const proof = await createAgentProof({
  policy: './agent.policy.json',           // or an inline object
  identity: new ENSIdentity({ chain: sepolia }),
  state: new GraphStateProvider({
    endpoint: process.env.GRAPH_ENDPOINT!,
    apiKey:   process.env.GRAPH_API_KEY!,
  }),
  approver: new LedgerApprover(),          // or ConsoleApprover for CI
  enforcement: {
    account: '0x…',                         // ERC-7579 smart account
    hook:    '0x…',                         // AgentPolicyHook
    bundler: process.env.BUNDLER_RPC!,
  },
  onDecision: (r) => dashboard.push(r),     // observability hook
});

const account = await proof.protect(smartAccountClient);
const result  = await account.execute(action);
```

`createAgentProof` is **async** because it verifies the ENS-published policy hash and warms Graph state before the first decision.

6.4 Evaluation order (deterministic, documented, tested)

1. `decode(action)` → `NormalizedIntent`. `UNKNOWN` kind ⇒ `BLOCK` unless `allowUnknownSelectors` is explicitly true.
2. `AllowlistPolicy` (contracts, then recipients) — cheapest, and catches T2.
3. `MaxTransactionPolicy`.
4. `MinBalancePolicy` (needs a balance read; cached per block).
5. `DailySpendPolicy` (needs Graph state; cached with a 15 s TTL and a hard read-through on `REQUIRE_APPROVAL`).
6. `ApprovalThresholdPolicy` — last, because it can only *upgrade* a passing action to `REQUIRE_APPROVAL`.

Rules: any `BLOCK` short-circuits and returns immediately with all violations found so far. `REQUIRE_APPROVAL` never overrides a `BLOCK`. `ALLOW` requires all policies to pass.

6.5 State management

```ts
export interface StateProvider {
  getDailySpend(account: Address, dayUtc: number): Promise<bigint>;
  getBalance(account: Address, asset: Address): Promise<bigint>;
  recordExecution(account: Address, intent: NormalizedIntent, txHash: Hex): Promise<void>;
}
```

Two implementations:

- `MemoryStateProvider` — tests and CI.
- `GraphStateProvider` — production/demo. Reads cumulative spend from our subgraph (§10.3). **Reconciles against the hook's onchain accumulator on every read**; if they disagree by more than one pending block, the SDK logs a `STATE_DIVERGENCE` warning and trusts the chain. Mention this in the demo — it shows you understand indexer lag.

6.6 Proof surfacing (a graded MVP criterion)

The build pipeline emits `proofs/*.json` from the SMT run. The SDK loads them at init and attaches the matching `ProofReference` to every `PolicyResult` whose deciding policy is formally verified. Result: a developer running the demo sees, in their terminal:

```
✖ BLOCK  swap 250.00 USDC via UniversalRouter
  └ maxTransaction: observed 250000000 > limit 100000000  (provenance: DECODED)
  └ invariant MAX_TRANSFER: PROVEN (solc-smtchecker, chc, 1,284 ms)
     proofs/MAX_TRANSFER.json
```

That single terminal block satisfies "counterexample/proof result visible in the developer workflow". Screenshot it for the submission.

---

7. Off-chain components

7.1 Decoder (`src/decode/`)

Module	Handles
`erc20.ts`	`transfer`, `transferFrom`, `approve`, `increaseAllowance`, Permit2 `approve`/`permitTransferFrom`
`uniswapV4.ts`	Universal Router `execute` — decode the command stream, extract input amounts and recipient
`native.ts`	Plain value transfers
`x402.ts`	Hedera payment payloads (§10.1)
`index.ts`	Registry: `selector → decoder`. Unknown ⇒ `kind: 'UNKNOWN'`

Decoders return `outflow` as **worst case**. For a v4 swap with `amountInMaximum`, use the maximum, never the quoted amount.

7.2 Agents (`apps/agents/`)

Two agents, both LangGraph, both thin — they exist to be constrained:

1. `trader/` — reads a price signal, proposes swaps. Deliberately given a prompt that makes it propose an oversized trade at step 5 of the demo. Do not fake this with an `if`; let the model actually do it, then show the block.
2. `researcher/` — needs paid data, pays the x402 service per query. Demonstrates that the same policy governs a payment as governs a swap.

7.3 Approval routing

```ts
export interface Approver {
  request(req: ApprovalRequest): Promise<{ approved: boolean; signature?: Hex; by: string }>;
}
```

- `ConsoleApprover` — y/n prompt. CI default.
- `DashboardApprover` — pushes to the dashboard over SSE, resolves on click.
- `LedgerApprover` — §10.5.

7.4 Dashboard (`apps/dashboard/`)

Next.js, split-pane, minimal white + `#0D9488` teal.

- **Left — "Agent reasoning"**: streamed LangGraph thoughts, proposed action, the agent's own justification.
- **Right — "AgentProof enforcement"**: the `NormalizedIntent`, per-policy pass/fail rows, the decision chip, the proof badge, and the tx hash or revert reason.
- **Footer**: daily-spend gauge (spent / limit), reconciled between Graph and chain.
- **Proof drawer**: renders `proofs/*.json`, including the counterexample from the deliberately-broken variant (§9.5). This is the screenshot that wins the "formal verification" criterion.

Constraint: the dashboard reads only. It never decides. If a judge asks "what happens if I close the dashboard", the answer is "nothing changes".

7.5 AgentProof Verification API (`packages/api/`)

The policy engine is exposed over HTTP as a small, stateless service. This is a second product surface, not a second product: every route is a thin wrapper over the same `PolicyEngine` the SDK uses, so there is exactly one implementation of every decision.

Why it exists

1. Agents not written in TypeScript (Python, Go, Rust, or an LLM calling tools directly) can use AgentProof.
2. It is the thing we sell per-call over x402 on Hedera (§10.1).
3. It is what a Bazantic gateway and Recipe are built on top of (§10.6).
4. It gives the dashboard and any external observer a read path into live policy state.

Routes

Method	Route	Body / params	Returns	Used by
`POST`	`/v1/verify`	`{ agent, action }`	`{ decision, reason, violations[], intent, proof }`	x402 service, Bazantic recipe, non-TS agents
`POST`	`/v1/decode`	`{ to, data, value, chainId }`	`NormalizedIntent` incl. worst-case `outflow`	Recipes, wallets, other teams' tooling
`GET`	`/v1/policy/{ensName}`	—	Live policy JSON + `policyHash`, resolved through ENSv2	ENS demo, external verifiers
`GET`	`/v1/spend/{account}`	`?asset=`	`{ dayUtc, spentGraph, spentOnchain, reconciled, limit }`	Dashboard, The Graph demo
`GET`	`/v1/proofs`	—	Current `ProofReference[]` from `proofs/*.json`	Dashboard proof drawer

Rules

- **Stateless and advisory.** The API never signs, never holds keys, and never executes. It answers questions.
- `/v1/verify` and `/v1/decode` are the two x402-gated routes; `/v1/policy`, `/v1/spend` and `/v1/proofs` are free reads.
- Deployed once, at a public HTTPS URL, from the same monorepo. One deploy target, not two.
- OpenAPI spec committed at `packages/api/openapi.yaml`. Bazantic and any judge can read it without running anything.

Framing warning. An HTTP "is this action safe?" endpoint is, on its own, exactly the application-level guardrail this project argues is bypassable. Always present it as **advisory L1 for agents that cannot embed the SDK**, with the onchain hook remaining the boundary. Say this in the README and in the video. A sharp judge will otherwise use the API against the thesis.

---

8. Onchain enforcement — `AgentPolicyHook.sol`

8.1 Why a hook, and not a standalone guard contract

The obvious alternative is a guard contract with a signature like `executeAction(agent, to, amount, data)` that the agent calls instead of its wallet. We reject it. It fails on three counts:

1. **It is an arbitrary-call proxy.** Anyone authorized can make the contract call anything with any calldata. If it ever holds funds, it is a honeypot.
2. **`amount` is a lie.** It is supplied by the same caller whose spending we are limiting, and it has no relationship to `data`. The invariant `amount <= maxTransaction` is proven, and enforces nothing.
3. **It cannot deliver the demo's key moment.** "Bypass the SDK and watch the wallet reject it" requires the *wallet* to enforce. A separate contract the agent can simply not call is not a boundary.

An ERC-7579 Type 4 Hook runs inside the account's own execution path. Every `execute` and `executeFromExecutor` passes through `preCheck` and `postCheck`. If the hook reverts, the whole UserOperation reverts. There is no path around it short of uninstalling the module, which requires the owner validator.

8.2 Interface (verify against the account implementation you install on)

```
interface IERC7579Hook {
    function preCheck(address msgSender, uint256 msgValue, bytes calldata msgData)
        external returns (bytes memory hookData);
    function postCheck(bytes calldata hookData, bool executionSuccess, bytes calldata executionReturnValue)
        external;
}
```

⚠ The `preCheck`/`postCheck` signatures changed during ERC-7579's life (older versions had `postCheck(bytes)` only). **Pin your account implementation and read its `HookManager` before writing the module.** Budget 2 hours for this; it is the single most likely place to lose half a day.

8.3 Contract

```
// contracts/src/AgentPolicyHook.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {PolicyLib} from "./lib/PolicyLib.sol";

/**
 * @title AgentPolicyHook
 * @notice ERC-7579 Type 4 hook enforcing AgentProof's critical policy subset
 *         inside a modular smart account's execution path.
 *
 * @dev Enforcement model:
 *      preCheck  — reject unknown targets/selectors, snapshot the tracked
 *                  asset balance, and return it as hookData.
 *      postCheck — measure the realised outflow (MEASURED provenance) and
 *                  apply PolicyLib.applySpend, which reverts if either
 *                  formally verified invariant would be broken.
 *
 *      Formally verified (see contracts/formal/PolicySpec.sol):
 *        MAX_TRANSFER : outflow_i <= maxTransaction, for all i
 *        DAILY_SPEND  : sum(outflow within a UTC day) <= dailyLimit
 */
contract AgentPolicyHook {
    using PolicyLib for PolicyLib.Window;

    uint256 internal constant MODULE_TYPE_HOOK = 4;

    struct Config {
        address asset;            // tracked ERC-20 (USDC on Sepolia)
        uint256 maxTransaction;   // base units
        uint256 dailyLimit;       // base units
        uint256 minBalance;       // base units
        bytes32 policyHash;       // keccak256 of the canonical policy JSON
        bool    installed;
    }

    mapping(address account => Config)            public config;
    mapping(address account => PolicyLib.Window)  public window;
    mapping(address account => mapping(address target => bool)) public allowedTarget;

    event PolicyInstalled(address indexed account, bytes32 policyHash);
    event SpendRecorded(address indexed account, uint256 amount, uint256 dayTotal);
    event TargetAllowed(address indexed account, address indexed target, bool allowed);

    error NotInstalled();
    error TargetNotAllowed(address target);
    error BelowMinBalance(uint256 balance, uint256 minBalance);
    error PolicyMismatch(bytes32 expected, bytes32 actual);

    // ------------------------------------------------------------------ module

    function onInstall(bytes calldata data) external {
        (Config memory c, address[] memory targets) = abi.decode(data, (Config, address[]));
        c.installed = true;
        config[msg.sender] = c;
        for (uint256 i; i < targets.length; ++i) {
            allowedTarget[msg.sender][targets[i]] = true;
            emit TargetAllowed(msg.sender, targets[i], true);
        }
        emit PolicyInstalled(msg.sender, c.policyHash);
    }

    function onUninstall(bytes calldata) external {
        delete config[msg.sender];
        delete window[msg.sender];
    }

    function isModuleType(uint256 t) external pure returns (bool) { return t == MODULE_TYPE_HOOK; }
    function isInitialized(address account) external view returns (bool) { return config[account].installed; }

    // ------------------------------------------------------------------ hook

    function preCheck(address, uint256, bytes calldata msgData)
        external
        returns (bytes memory hookData)
    {
        Config memory c = config[msg.sender];
        if (!c.installed) revert NotInstalled();

        // Decode the 7579 execution to recover the ultimate target(s).
        address target = _decodeTarget(msgData);
        if (!allowedTarget[msg.sender][target]) revert TargetNotAllowed(target);

        uint256 balanceBefore = IERC20(c.asset).balanceOf(msg.sender);
        return abi.encode(balanceBefore, uint64(block.timestamp / 1 days));
    }

    function postCheck(bytes calldata hookData, bool, bytes calldata) external {
        Config memory c = config[msg.sender];
        (uint256 balanceBefore, uint64 today) = abi.decode(hookData, (uint256, uint64));

        uint256 balanceAfter = IERC20(c.asset).balanceOf(msg.sender);
        uint256 outflow = balanceBefore > balanceAfter ? balanceBefore - balanceAfter : 0;

        if (outflow > 0) {
            // Reverts on MAX_TRANSFER or DAILY_SPEND violation.
            PolicyLib.Window memory next = PolicyLib.applySpend(
                window[msg.sender],
```

```
                PolicyLib.Limits(c.maxTransaction, c.dailyLimit),
                outflow,
                today
            );
            window[msg.sender] = next;
            emit SpendRecorded(msg.sender, outflow, next.spent);
        }

        if (balanceAfter < c.minBalance) revert BelowMinBalance(balanceAfter, c.minBalance);
    }

    // ------------------------------------------------------------ internals

    /// @dev Decodes an ERC-7579 execution payload to its target address.
    ///      Supports CALLTYPE_SINGLE in the MVP; batch handling reverts.
    function _decodeTarget(bytes calldata msgData) internal pure returns (address) { /* … */ }
}
```

8.4 Design notes worth defending out loud

Choice	Rationale
Outflow is a **balance delta**, not a parameter	Defeats T4. An approval that is immediately drained inside the same UserOp is caught; a raw approval that is drained *later* is caught by the target allowlist plus the SDK's worst-case approval rule
Allowlist checked in `preCheck`	Cheap rejection before any state change; also the anti-prompt-injection control
Accumulator written in `postCheck`	Guarantees we count what actually happened, not what was intended
`minBalance` in `postCheck`	A floor is only meaningful after the fact
Batch executions revert in the MVP	Honest scope limit. A batch that splits a spend across calls is a real bypass; rejecting batches is the safe default. Document it
Approvals to non-allowlisted spenders revert	Closes the "approve now, drain next block" hole for the demo's threat set

8.5 Known gaps to state in the README (do not hide these)

1. **Delegatecall executions** are not analysed; the account should disallow them for the agent validator.
2. **Multi-asset spend** is not tracked; only the configured asset. A swap that spends WETH instead of USDC is bounded only by the allowlist.
3. **Cross-UserOp approval draining** by an allowlisted target is out of scope.
4. Batch mode is unimplemented.

Each of these has a one-line mitigation in the demo config (single asset, single allowlisted router, no batching). Listing them is a strength: it shows you know where the boundary of your own proof is.

8.6 Deployment set (Sepolia)

Contract	Purpose
`AgentPolicyHook`	The enforcement module
`MockUSDC` (6 dec)	Only if a suitable Sepolia USDC faucet is unavailable — prefer real testnet USDC
ERC-7579 account	**Use an existing implementation.** Do not write an account
`AgentSubnameRegistrar`	§10.2, ENSv2

Deploy script must: deploy hook → deploy/import account → install validator (agent session key) → install hook with encoded `Config` → verify on Etherscan → write addresses to `deployments/sepolia.json`, which both the SDK and the dashboard read. No address is ever hardcoded in two places.

9. Formal verification plan

9.1 Scope statement (put this verbatim in `docs/formal-verification.md`)

We formally verify the **deterministic policy arithmetic** that governs onchain enforcement. We do not verify the LLM, the SDK, or the external protocols. The proof holds for the modeled system: the `PolicyLib` state machine, unbounded `uint256` arithmetic with Solidity 0.8 checked semantics, and an arbitrary adversarial sequence of `applySpend` calls.

9.2 Why we verify a library, not the hook

SMTChecker's CHC engine loses precision fast on contracts with external calls, delegatecall, and cross-contract mappings — exactly what a 7579 hook is full of. It will return `unknown` and we will have burned a day. So:

```
PolicyLib.sol        ← pure functions + explicit state struct   ← VERIFIED HERE
    ▲
    │ used by
AgentPolicyHook.sol  ← external calls, balances, account glue   ← fuzzed, not proven
```

`PolicyLib` contains no external calls, no dynamic arrays, and one mapping keyed by `(account, day)` that we model as a struct parameter. This is the difference between a proof and a timeout.

9.3 The library

```
// contracts/src/lib/PolicyLib.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

library PolicyLib {
    struct Limits {
        uint256 maxTransaction;   // per-op ceiling, USDC base units
        uint256 dailyLimit;       // per-UTC-day ceiling
    }

    struct Window {
        uint64  day;              // block.timestamp / 1 days at last write
        uint256 spent;            // cumulative spend within `day`
    }

    error ExceedsMaxTransaction(uint256 amount, uint256 limit);
    error ExceedsDailyLimit(uint256 wouldBe, uint256 limit);

    /// @notice Pure transition. Reverts iff the spend is not permitted.
    /// @dev The two assertions below are the formally verified properties.
    function applySpend(
        Window memory w,
        Limits memory l,
        uint256 amount,
        uint64  today
    ) internal pure returns (Window memory next) {
        if (amount > l.maxTransaction) revert ExceedsMaxTransaction(amount, l.maxTransaction);

        uint256 base = (w.day == today) ? w.spent : 0;   // UTC-day reset
        uint256 wouldBe = base + amount;                  // 0.8.x checked add
        if (wouldBe > l.dailyLimit) revert ExceedsDailyLimit(wouldBe, l.dailyLimit);

        next = Window({ day: today, spent: wouldBe });

        // --- Verified properties ------------------------------------------
        assert(amount <= l.maxTransaction);          // P_MAX_TRANSFER
        assert(next.spent <= l.dailyLimit);          // P_DAILY_SPEND
        assert(next.day == today);                   // P_WINDOW_MONOTONIC
        // ------------------------------------------------------------------
    }
}
```

9.4 Harness and invocation

```
// contracts/formal/PolicySpec.sol
pragma solidity ^0.8.28;
import {PolicyLib} from "../src/lib/PolicyLib.sol";

/// @dev SMTChecker explores arbitrary sequences of `step` calls.
contract PolicySpec {
    using PolicyLib for PolicyLib.Window;
    PolicyLib.Window  private w;
    PolicyLib.Limits  private l;

    constructor(uint256 maxTx, uint256 daily) {
        require(maxTx > 0 && daily >= maxTx);
        l = PolicyLib.Limits(maxTx, daily);
    }

    function step(uint256 amount, uint64 today) external {
        require(today >= w.day);                     // time moves forward
        w = PolicyLib.applySpend(w, l, amount, today);

        // Inductive invariants over the whole reachable state space:
        assert(w.spent <= l.dailyLimit);
        assert(amount  <= l.maxTransaction);
    }
}
```

```
        # scripts/run-formal-verification.sh
        solc --model-checker-engine chc \
             --model-checker-targets assert,overflow,underflow \
             --model-checker-timeout 120000 \
             --model-checker-show-unproved \
             --model-checker-invariants contract,reentrancy \
             --model-checker-contracts contracts/formal/PolicySpec.sol:PolicySpec \
             contracts/formal/PolicySpec.sol \
          | tee artifacts/smt/raw.log

        node scripts/parse-smt-output.mjs artifacts/smt/raw.log > proofs/summary.json
```

`parse-smt-output.mjs` turns solc's text into the `ProofReference` JSON the SDK and dashboard consume. Write it early — it is 40 lines and it is what makes the proof *visible*, which is a graded criterion.

9.5 The counterexample demo (do not skip this)

Keep `contracts/formal/PolicySpecBroken.sol`, identical except the daily check uses `>=` instead of `>` on a boundary, or omits the `w.day == today` reset guard. Run it in CI. SMTChecker produces a concrete counterexample trace. Render that trace in the dashboard's proof drawer next to the passing one.

Effect on judges: it proves the tool is actually running rather than being cited. "Here is the property; here is what the solver says when we break it" is worth more than two green checkmarks.

9.6 Complementary testing

Layer	Tool	Target
Pure logic	SMTChecker CHC	The two invariants, exhaustively
Stateful hook	Foundry invariant testing (`invariant_*` + handler with bounded actors)	`sum(outflows in day) ≤ dailyLimit` against the real hook + a mock ERC-20
Decoders	Vitest + fixtures from real Sepolia calldata	Decoded amount == measured delta
Differential	Fuzz the same input through `PolicyEngine` (TS) and `PolicyLib` (Solidity)	L1 and L2 never disagree on ALLOW/BLOCK

The differential test is the sleeper feature. "Our off-chain engine and our on-chain hook are fuzz-tested to agree" is a strong, specific, checkable claim.

9.7 Honest limitations (in the README)

- Proves the spec we wrote, not "the system is safe".
- SMTChecker may return `unknown`; we report `UNPROVEN` rather than claiming success.
- The hook itself is fuzzed, not proven. Say so.
- Day-boundary behavior (T6) is a known accepted property, not a bug.

---

10. Sponsor integrations

Each subsection ends with a **qualification checklist** taken from the published ETHOnline 2026 requirements. Treat these as acceptance tests, not aspirations. Re-read the live prize page before submitting — sponsors edit requirements mid-event.

10.0 Portfolio summary

Sponsor	Track	Open pool	Our fit	Effort	Priority
Hedera	AI & Agentic Payments	$6,000 (up to 3 × $2,000)	High — **once we host the service**	2 days	P0
The Graph	Best AI Tooling or AI Use Case (From Scratch)	$5,000	High	1.5 days	P0
ENS	Best Use of ENSv2	$4,500	High — **only with a real ENSv2 registry**	1.5 days	P0
Uniswap	Best Uniswap Stack Contribution	$3,000 (up to 3 × $1,000)	Medium	1 day	P1
Ledger	AI Agents × Ledger	$3,500	High conceptually, gated on hardware	0.5–1 day	P1
Bazantic	Best Recipe using sponsor APIs	$1,000	Cheap add-on once the API exists	3 hours	P2
Bazantic	Agentify a new API	$1,000	Possible, needs one genuinely new API	3 hours	P3
Chainlink	Best Confidential Workflow	$2,500	Thematic, heavy	2 days	P3 (skip)

Realistic target: **4 strong integrations + 2 opportunistic**. P0 and P1 are the project. P2 and P3 are attempted only after Gate G4 (§15) and only because the Verification API (§7.5) makes them nearly free. Half-finished integrations lose to complete ones every time.

10.1 Hedera — host the x402-gated verification service (P0)

The strategic move. The bounty does not reward an agent that merely *pays* for something. It requires you to stand up a real x402-gated service on Hedera and build the platform that consumes it.

So: **the AgentProof verification engine itself becomes the paid service.**

```
POST https://verify.agentproof.dev/v1/verify
  → 402 Payment Required  { asset: USDC (HTS), amount: 0.01, network: hedera-testnet,
                            payTo: 0.0.xxxx, facilitator: blocky402 }
  ← client constructs + partially signs payment, retries with X-PAYMENT header
  → facilitator verifies + settles (facilitator pays gas)
  → 200 { decision, reason, violations, proof }
```

Why this is the right service to sell:

- It is genuinely metered per call — one verification, one micropayment. Not a flat subscription wrapper.
- It makes AgentProof usable by agents that are not in TypeScript, which is a real product argument.
- It closes a beautiful loop: **an agent pays, under AgentProof policy, for an AgentProof verification.** The payment itself is

policy-checked. That is a memorable 20 seconds of demo.

Build plan

Component	Detail
Server	The Verification API of §7.5 (Hono/Express), with `x402` middleware in front of `/v1/verify` and `/v1/decode`, deployed at a public HTTPS URL
Facilitator	**Blocky402** (required). Testnet base URL is published by Blocky402 — verify the `/supported` endpoint returns `hedera-testnet` before building on it
Settlement asset	USDC via HTS, or HBAR. Hedera's scheme uses partially signed transactions where the facilitator pays gas and submits
Client	`packages/x402-client` used by the `researcher` agent and by `HederaEnforcer` in the SDK
Audit trail	Write each paid verification's `{policyHash, decision, amountPaid}` to an **HCS topic**. Explicit "extra points" item and it costs ~20 lines
Identity	Publish the agent's identity as **ERC-8004** / **HCS-14** if time allows — another listed extra-points item

Do not build on an unverified npm wrapper. If a third-party SDK such as `@clawpay-hedera/sdk` does not settle through Blocky402, it does not satisfy the requirement. Use the official `x402` packages plus `@hashgraph/sdk` and point the facilitator config at Blocky402.

Qualification checklist

- ☐ Live x402-gated service on Hedera testnet, settled through Blocky402
- ☐ A platform/agent that consumes it and completes ≥1 real paid request end to end
- ☐ Public repo with README covering setup, architecture, and the payment flow
- ☐ Demo video ≤ 5 minutes showing the paid request executing
- ☐ Extra points: per-call metering ✅, HCS audit trail ✅, ERC-8004/HCS-14 identity ☐, HTS settlement ✅, agent discovery ☐

10.2 ENS — agent identity as a namespace (P0)

What will not qualify: a plain `client.getEnsAddress({ name: 'agent.eth' })` lookup on mainnet. The bounty requires ENSv2 features to be central rather than a cosmetic add-on, and the demo must not use hard-coded values. Name resolution alone is a cosmetic add-on.

What we build: we own `agentproof.eth` (Sepolia ENSv2 beta) and deploy **our own subname registry** so that every protected agent is a namespace with its own identity, its own records, and delegated permissions.

```
agentproof.eth                      ← parent name, our Permissioned Registry
 └── trader.agentproof.eth          ← per-agent subname (ERC-1155 token)
      ├── addr                      → the agent's ERC-7579 smart account
      ├── text "agentproof.policy"  → policyHash (keccak256 of the canonical JSON)
      ├── text "agentproof.hook"    → deployed AgentPolicyHook address
      └── text "agentproof.status"  → active | suspended
```

Enhanced Access Control is the star of the integration. EAC supports per-resource roles, including granting an account the right to edit only certain records. We use that to encode the entire trust model in ENS itself:

Holder	Role granted	On resource	Meaning
Owner (human/Ledger)	record-edit on `agentproof.policy`	the agent's name	Only a human can change the policy pointer
Agent session key	record-edit on `agentproof.status` only	the agent's name	The agent may mark itself suspended, never widen its own policy
Registrar contract	`ROLE_REGISTRAR` (`1 << 0`), `ROLE_RENEW` (`1 << 16`)	`ROOT_RESOURCE`	Can mint/renew agent subnames

That table *is* the AgentProof thesis expressed in ENS permissions: **the agent can act inside its namespace but cannot rewrite the namespace's rules.** It is exactly the "agents as namespaces, each with their own identity and permissions" bonus the bounty calls out.

Build plan

1. Register `agentproof.eth` on the ENSv2 Sepolia beta (`app.ens.dev` / `explorer.ens.dev`).
2. Deploy `AgentSubnameRegistrar.sol` following the ENSv2 contract-developer tutorial: registrar sits in front of the registry, registry stores names and enforces permissions.
3. Deploy a Permissioned Resolver for the agent names; write `addr` + the three text records.
4. Grant EAC roles per the table above.
5. SDK `ENSIdentity`: resolve `trader.agentproof.eth` through the Universal Resolver on Sepolia, read the `agentproof.policy` text record, compare with the local policy hash, **refuse to start on mismatch**. Also resolve `agentproof.hook` and assert the deployed hook matches.
6. Demo moment: an operator tries to widen the policy using the *agent's* key. The EAC role check reverts. Then the owner key does it and it succeeds.

Notes

- ENSv2 contracts are beta and "not yet final". Pin versions and record the exact addresses used in `deployments/ensv2-sepolia.json`.
- Use an ENSv2-ready viem/ENSjs version. ENSjs is the only one of the four supported libraries with purpose-built write helpers, so prefer it for record writes.
- The `ens-cli` is agent-native and useful for scripting the setup, but it is explicitly not production-ready — use it for ops, not inside the SDK.
- Relevant reading: ENSIP-25 (AI agent registry name verification) and ENSIP-26 (agent text records). If our record names can align with ENSIP-26 conventions, do that — it turns a custom hack into standards use.

Qualification checklist

- [ ] Built on ENSv2, Sepolia
- [ ] ENSv2 features central: hierarchical registry ✓, own subname registry ✓, Permissioned Resolver ✓, EAC delegation ✓
- [ ] Functional demo, no hard-coded values (resolution is live, records are read at runtime)
- [ ] Open-source repo + video and/or live demo link

10.3 The Graph — live historical state for stateful policy (P0)

Track: *Best AI Tooling or AI Use Case with The Graph — From Scratch* ($5,000). We are net-new, so we file under Start Fresh.

Do not target the composability track unless we do more: it explicitly excludes simply querying one subgraph, and requires composing two or more Graph products or building on a standardized schema.

What we build

1. **`subgraphs/agent-history/`** — indexes our hook's `SpendRecorded`, `PolicyInstalled`, `TargetAllowed` events plus ERC-20 `Transfer`s to/from protected accounts. Deploy via **Subgraph Studio**; query with a real API key. Entities: `Agent`, `PolicyInstall`, `Execution`, `DailyAggregate`.
2. **`GraphStateProvider`** in the SDK — the daily-spend policy reads cumulative spend from this subgraph and reconciles it with the hook's onchain accumulator (§6.5). This is the "meaningful work with the data" the bounty demands: the query result *decides* whether a transaction happens.
3. **Cross-protocol read** — before allowing a swap, query a public Uniswap subgraph for the pool's recent state and reject actions against a pool with implausible liquidity. Cheap, and it turns one subgraph into two.
4. **x402-paid queries** — the bounty explicitly mentions letting your agent pay per query autonomously with x402. We already have x402 plumbing from §10.1. Wiring the agent to pay for a data query ties the two sponsors into one flow rather than two features.
5. Consider the **Agent0 / ERC-8004 subgraphs** for agent identity/reputation reads — directly on-theme for an agent-safety product.

Hard requirement: live data from a Graph provider. Mocked, local-only, or static datasets disqualify. Never demo from a fixture. Keep a recorded fallback video, but the live path must be real.

Qualification checklist

- [ ] The Graph is load-bearing: the agent's spend decision depends on it
- [ ] Live data via Subgraph Studio with an API key
- [ ] Meaningful work with the data (a policy decision, not a printed query result)
- [ ] Open source + clear README so judges can run it
- [ ] Public repo + 2–4 minute demo video
- [ ] Correct pool selected: **Start Fresh**

10.4 Uniswap — the real DeFi action being protected (P1)

Track: *Best Uniswap Stack Contribution* ($3,000, up to 3 teams × $1,000).

The bounty is broad: build on or integrate any part of the Uniswap stack, including v4 hooks, the API, or tooling for the ecosystem. Our angle is **tooling**: `@agentproof/sdk` is reusable infrastructure that lets any agent trade on Uniswap under provable constraints.

Build plan

- The `trader` agent swaps USDC → WETH on Uniswap v4 (Sepolia) through the protected account.
- Write `src/decode/uniswapV4.ts` to decode Universal Router command streams to `(tokenIn, amountInMaximum, recipient)`. This decoder is the actual contribution: it is what makes v4 calldata policy-checkable, and nobody else ships it.
- Publish it as a standalone export (`@agentproof/sdk/decode/uniswap`) so it is reusable outside our product.
- **Look up current v4 deployment addresses in the Uniswap docs deployments page.** Do not copy addresses from memory or from an old repo.

Qualification checklist — do not lose the prize on paperwork

- [ ] Public GitHub repo, open-source
- [ ] **`FEEDBACK.md` in the repo** (required)
- [ ] **Uniswap Developer Feedback Form submitted, including the link to FEEDBACK.md** (required)
- [ ] README points to the exact contracts and lines of code implementing the integration

Assign the feedback form to a named person on Day 11. Submissions without it are audited and may be dropped.

10.5 Ledger — the human in the loop (P1, high EV)

The Ledger AI-agents track asks for agents that pay for what they use, systems that require a human before anything irreversible, and products that make autonomous behavior safer rather than bypassing user intent. Our `REQUIRE_APPROVAL` decision is that bullet, already in the policy model.

Minimal integration: `LedgerApprover` implements the `Approver` interface. When an action exceeds `approvalThreshold` but is otherwise valid, the SDK renders the decoded intent (recipient, asset, amount — human-readable, not raw calldata) and requests a device confirmation. The owner-signed UserOp uses the owner validator, which carries the elevated ceiling. The agent's session key can never produce that signature.

Stretch: move the agent's session key and API secrets onto the **Ledger Key Ring CLI** (`wallet-cli ring`), so the agent process holds scoped capabilities rather than raw keys. This addresses the "agents that use secrets they cannot leak" ask, and it is architecturally coherent with our thesis.

Gate: this requires a physical device and the Ledger Agent Stack. Decide by **12 September**. If nobody on the team has a device by then, drop it and keep `DashboardApprover` — do not half-ship it.

10.6 Bazantic — two tracks off one API (P2 / P3)

Both open Bazantic tracks are $1,000 pools and both are built on the same prerequisite: an account at `bazantic.com` and an x402/MPP gateway for our API. Because §7.5 already gives us a documented HTTP surface, the marginal cost is small. Do them in order; the second is optional.

Shared prerequisites (≈45 minutes, do once)

- Create the Bazantic account. Record the username (email or GitHub handle) — it must be included in every submission for attribution.
- Create an x402/MPP Gateway in Bazantic pointing at the deployed AgentProof Verification API.

Track A — Best Recipe that uses ETHGlobal sponsor APIs (P2, ≈2 hours)

Requires a Recipe that chains our service with at least one other service already on Bazantic or offered by an ETHOnline sponsor, where the final result depends meaningfully on both.

Our recipe: **"Verify before you swap."**

```
Uniswap API  GET /swap        → returns route + calldata
      ↓
AgentProof   POST /v1/decode  → normalized intent, worst-case outflow
      ↓
AgentProof   POST /v1/verify  → ALLOW / BLOCK / REQUIRE_APPROVAL + reason
      ↓
agent executes, or stops and explains why
```

Both services are load-bearing: without Uniswap there is no calldata, without AgentProof there is no decision. Record the full flow start to finish on screen.

Track B — Agentify a new API (P3, ≈3 hours, only if Track A lands early)

Requires adding a service that was **not** available through Bazantic and is **not** offered by any ETHOnline sponsor at the start of the event, then using it in a recipe alongside our own.

Candidate: a public **token-risk / contract-metadata API** (honeypot detection, verified-source and ownership checks, blocklists). It is thematically exact — AgentProof's allowlist policy is precisely where such a signal belongs — and it produces a recipe other builders would genuinely reuse:

```
Token risk API → risk verdict for tokenOut
      ↓
AgentProof /v1/verify → policy decision that now includes counterparty risk
```

Do not attempt Track B if it means writing a new policy type under deadline. Feed the risk verdict in as metadata on the existing allowlist policy, or skip the track.

Qualification checklist (both tracks)

- [ ] Bazantic account created; **username recorded in the submission**
- [ ] x402/MPP Gateway created for the AgentProof API
- [ ] Recipe explains when, why, and how to use the service
- [ ] Final result depends meaningfully on both services
- [ ] Screen recording showing the completed task start to finish
- [ ] Track B only: the added API was genuinely unavailable on Bazantic and from all sponsors at event start

10.7 Sponsors we are declining, and why

Sponsor	Why not
1inch ($7,000)	The bounty is Aqua/SwapVM-specific and requires official Aqua/SwapVM contracts with onchain execution. It is not a substitute for Uniswap; it is a separate build
World ($7,000)	AgentKit is Continuity-only. Selfie Check is open but is a biometric-credential product with no honest tie to policy enforcement
Arc ($10,000)	Genuinely tempting — the agentic-economy track wants agents that hold wallets, make payments, and manage risk. But it means a second chain, Circle Agent Stack, and a mainnet-readiness expectation by 30 September. Revisit only if the Hedera track collapses
Privy ($5,000)	Overlaps our own product (policies, quorum approvals, signers). **Prepare an answer for judges:** Privy enforces policy in its infrastructure; AgentProof enforces it in the account, so it survives a compromised backend and is verifiable by anyone onchain. Different trust assumptions, not a competitor
Chainlink ($2,500)	CRE Confidential Workflows fit thematically ("privacy-preserving risk assessment and policy enforcement") but require a TEE workflow to be core functionality. Too heavy for the remaining budget

11. Repository structure

```
agentproof/
├── apps/
│   ├── dashboard/                  # Next.js split-pane demo UI
│   └── agents/
│       ├── trader/                 # LangGraph DeFi agent (Uniswap v4)
│       └── researcher/             # agent that buys data via x402
├── packages/
│   ├── sdk/                        # @agentproof/sdk  ← THE PRODUCT
│   │   ├── src/
│   │   │   ├── index.ts
│   │   │   ├── core/               # AgentProof.ts, PolicyEngine.ts, types.ts
│   │   │   ├── policies/           # MaxTransaction, DailySpend, Allowlist,
│   │   │   │                       # MinBalance, ApprovalThreshold
│   │   │   ├── decode/             # erc20, uniswapV4, native, x402, registry
│   │   │   ├── identity/           # ENSIdentity (ENSv2, Sepolia)
│   │   │   ├── state/              # MemoryStateProvider, GraphStateProvider
│   │   │   ├── enforcement/        # SmartAccountClient glue, UserOp builder
│   │   │   ├── approval/           # Console, Dashboard, Ledger approvers
│   │   │   ├── proofs/             # loads proofs/*.json → ProofReference
│   │   │   └── utils/              # logger, errors, units (usdc())
│   │   └── tests/{unit,integration,fixtures}/
│   ├── api/                        # AgentProof Verification API (§7.5)
│   │   ├── src/routes/             # verify, decode, policy, spend, proofs
│   │   ├── src/x402/               # Blocky402 middleware, HCS audit writer
│   │   └── openapi.yaml            # published spec (Bazantic + judges read this)
│   ├── x402-client/                # shared payment client
│   └── config/                     # tsconfig/eslint/tsup base configs
├── contracts/
│   ├── src/
│   │   ├── AgentPolicyHook.sol
│   │   ├── AgentSubnameRegistrar.sol
│   │   └── lib/PolicyLib.sol
│   ├── formal/
│   │   ├── PolicySpec.sol          # SMTChecker harness (proves)
│   │   └── PolicySpecBroken.sol    # deliberately broken (counterexample)
│   ├── test/
│   │   ├── AgentPolicyHook.t.sol
│   │   ├── invariant/              # Foundry invariant + handler
│   │   └── integration/            # 7579 account + hook end-to-end
│   ├── script/Deploy.s.sol
│   └── foundry.toml
├── subgraphs/agent-history/        # schema.graphql, subgraph.yaml, mappings
├── proofs/                         # generated: MAX_TRANSFER.json, DAILY_SPEND.json
├── deployments/                    # sepolia.json, hedera-testnet.json, ensv2-sepolia.json
├── docs/
│   ├── threat-model.md
│   ├── formal-verification.md
│   └── quickstart.md
├── scripts/
│   ├── run-formal-verification.sh
│   ├── parse-smt-output.mjs
│   └── demo-runner.ts              # scripted end-to-end demo, idempotent
├── .github/workflows/{ci.yml,formal-verify.yml}
├── FEEDBACK.md                     # Uniswap requirement
├── README.md
├── pnpm-workspace.yaml
└── turbo.json
```

Two rules: `deployments/*.json` **is the single source of truth for addresses**, and `proofs/*.json` **is generated, never hand-edited**.

12. Environment and configuration

```
# ---- EVM (Sepolia) ----
SEPOLIA_RPC_URL=
BUNDLER_RPC_URL=                # ERC-4337 bundler with 7579 account support
OWNER_PRIVATE_KEY=              # human/owner key — NEVER in the agent process
AGENT_SESSION_KEY=              # scoped session key the agent signs with
SMART_ACCOUNT_ADDRESS=
AGENT_POLICY_HOOK_ADDRESS=
USDC_ADDRESS=

# ---- ENSv2 (Sepolia) ----
ENS_PARENT_NAME=agentproof.eth
AGENT_SUBNAME=trader.agentproof.eth
ENS_REGISTRAR_ADDRESS=
ENS_RESOLVER_ADDRESS=

# ---- The Graph ----
GRAPH_ENDPOINT=                 # Subgraph Studio query URL
GRAPH_API_KEY=
UNISWAP_SUBGRAPH_ENDPOINT=

# ---- Hedera ----
HEDERA_ACCOUNT_ID=0.0.xxxxx
HEDERA_PRIVATE_KEY=             # ECDSA
HEDERA_NETWORK=testnet
X402_FACILITATOR_URL=           # Blocky402 testnet base URL — verify /supported
X402_SERVICE_URL=               # our deployed verification endpoint
HCS_AUDIT_TOPIC_ID=

# ---- Verification API ----
API_PUBLIC_URL=                 # public HTTPS base URL of packages/api
X402_PRICE_VERIFY_USDC=0.01     # per-call price for /v1/verify
X402_PRICE_DECODE_USDC=0.005    # per-call price for /v1/decode

# ---- Ledger (optional) ----
LEDGER_APPROVAL_ENABLED=false
```

Key hygiene is part of the pitch. The agent process gets `AGENT_SESSION_KEY` only. `OWNER_PRIVATE_KEY` lives in the approval service (or on the Ledger). If a judge greps your repo and finds the agent holding the owner key, the entire thesis dies. Add a CI check that fails if `OWNER_PRIVATE_KEY` is read anywhere under `apps/agents/`.

---

13. Testing and CI

13.1 Test matrix

Level	Tool	What it proves	Gate
Unit — policies	Vitest	Each policy's ALLOW/BLOCK boundary, including off-by-one at the limit	100% branch coverage on `policies/`
Unit — decoders	Vitest + real calldata fixtures	Decoded outflow matches the on-chain measured delta	Every supported selector has a fixture
Contract	Foundry	Hook reverts on each violation; accumulator correctness; day reset	All green
Invariant	Foundry `invariant_*` + bounded handler	`sum(day outflow) ≤ dailyLimit` under random action sequences	10,000 runs, depth 50
Formal	solc SMTChecker CHC	`MAX_TRANSFER`, `DAILY_SPEND`	`PROVEN`, or the build reports `UNPROVEN` honestly
Differential	Vitest + forge fixture export	TS engine and Solidity library agree on every generated case	Zero disagreements
Integration	Foundry + local 7579 account	SDK-signed UserOp succeeds; hand-signed over-limit UserOp reverts	Both assertions pass
E2E	`scripts/demo-runner.ts` against Sepolia + Hedera testnet	The full nine-step demo	Green twice consecutively before recording

13.2 CI

- `ci.yml` — install, build, lint, typecheck, vitest, `forge test`, `forge coverage`.
- `formal-verify.yml` — runs the SMT script, uploads `proofs/*.json` as artifacts, and **fails the build if `MAX_TRANSFER` regresses from `PROVEN`**. Also runs `PolicySpecBroken` and asserts a counterexample is produced (a green build there would mean the checker is not actually running).

13.3 The one test that wins arguments

```
        it('rejects a UserOp that bypasses the SDK entirely', async () => {
          const userOp = await handSignOverLimitUserOp(sessionKey, 250_000_000n); // no SDK
          await expect(bundler.send(userOp)).rejects.toThrow(/ExceedsMaxTransaction/);
        });
```

Put this test's name in the README. It is the difference between a linter and a safety runtime.

---

14. Demo script

Total: **4 minutes** for the main video (Hedera requires ≤5, The Graph wants 2–4). Cut a 2-minute version for The Graph submission from the same footage.

14.1 Setup shown on screen

Agent `trader.agentproof.eth` · smart account funded with 1,000 USDC · policy: max transaction 100 USDC, daily spend 500 USDC, approval threshold 100 USDC, min balance 10 USDC, Uniswap v4 router allowlisted.

14.2 Shot list

#	Time	Beat	What the viewer sees	Sponsor proved
1	0:00	**The claim**	Title card: "Trust the agent to decide. Don't trust it to enforce its own limits."	—
2	0:12	**Identity**	Terminal resolves `trader.agentproof.eth` via ENSv2 on Sepolia; reads the `agentproof.policy` text record; hash matches the local policy. Show the ENS explorer page with the subname and its records	ENS
3	0:35	**Valid action**	Agent reasons about a price signal, proposes an $80 USDC → WETH swap. Right pane: five policy rows go green. Daily-spend row shows the figure fetched **live from the subgraph**	The Graph, Uniswap
4	0:55	**Execution**	UserOp lands. Etherscan tx. Daily gauge moves to 80/500	Uniswap
5	1:15	**Invalid action**	Same agent proposes a $250 swap. `BLOCK`. Reason: `maxTransaction: observed 250000000 > limit 100000000 (DECODED)`	—
6	1:35	**The proof**	Proof drawer opens: `MAX_TRANSFER — PROVEN (solc SMTChecker, CHC, 1.2 s)`. Then flip to the broken variant and show the **counterexample trace** the solver produced	Formal verification criterion
7	2:05	**The bypass** ⭐	Close the SDK. Hand-sign a UserOp for 250 USDC directly with the agent session key. Bundler returns `ExceedsMaxTransaction`. Etherscan shows the revert. "*The SDK is convenience. This is the boundary.*"	Core thesis
8	2:35	**Human in the loop**	A 120 USDC action returns `REQUIRE_APPROVAL`; approval request appears; Ledger device (or dashboard) shows decoded human-readable intent; owner approves; it executes under the owner validator	Ledger
9	3:00	**Agentic payment** ⭐	The `researcher` agent hits the x402-gated AgentProof verification service on Hedera. 402 → payment → 200. **The payment itself is policy-checked by the same hook.** Show the HashScan transaction and the HCS audit message	Hedera
10	3:35	**Tamper attempt**	Operator tries to raise the daily limit using the agent's key. ENSv2 EAC role check reverts. Owner key succeeds	ENS
11	3:50	**Close**	One screen: `npm install @agentproof/sdk`, six lines of config, and the line "the AI can act autonomously, but it cannot redefine the boundaries within which it acts"	—

14.3 Recording rules

- Record on **15 September**, not the 16th. Leave a full day of slack.
- `scripts/demo-runner.ts` must be idempotent and re-runnable: it resets the daily accumulator via a fresh account or a time warp, and reuses deployed addresses from `deployments/`.
- Record one clean full-length take with everything live. Then record backup takes of shots 7 and 9 (the two that depend on external infrastructure).
- No cuts inside shot 7. A cut there reads as a fake.

---

15. Execution plan — 4 to 16 September

Two workstreams. If you are a team of two: **A = contracts + verification**, **B = SDK + integrations**. If solo, follow the same order and cut §10.5 and §10.6 immediately.

Phase 1 — Foundations (4–6 Sept)

Day	A (chain)	B (TypeScript)
Thu 4	Turborepo + Foundry scaffold; pick and pin the ERC-7579 account implementation; **read its HookManager and confirm the preCheck/postCheck signatures**	SDK skeleton, `types.ts`, `PolicyEngine`, all six policies with unit tests
Fri 5	`PolicyLib.sol` + `AgentPolicyHook.sol` first pass; local Foundry test installing the hook on an account	`decode/erc20.ts` + `decode/native.ts` with real calldata fixtures; `MemoryStateProvider`
Sat 6	Deploy hook + account to Sepolia; hand-signed over-limit UserOp reverts	`createAgentProof` → `protect` → `execute` working end-to-end against Sepolia

Gate G1 (end of 6 Sept): a hand-signed over-limit UserOp reverts onchain. If this is not true, stop everything else and fix it — the entire project rests on it.

Phase 2 — Proof + identity (7–9 Sept)

Day	A	B
Sun 7	`PolicySpec.sol` harness; first SMTChecker run; tune until `PROVEN`	`parse-smt-output.mjs`; `proofs/` loader; proof rendering in the terminal
Mon 8	`PolicySpecBroken.sol` producing a real counterexample; Foundry invariant suite	ENSv2: register `agentproof.eth` on Sepolia, deploy the subname registrar
Tue 9	Differential fuzz TS ↔ Solidity	ENSv2 Permissioned Resolver + text records + EAC role grants; `ENSIdentity` refuses to start on hash mismatch

Gate G2 (end of 9 Sept): `MAX_TRANSFER` and `DAILY_SPEND` report `PROVEN`, and the broken variant emits a counterexample. If SMTChecker returns `unknown`, simplify `PolicyLib` further (drop `minBalance` from the verified scope, shrink integer widths) rather than pushing the deadline.

Phase 3 — Data + payments (10–12 Sept)

Day	A	B
Wed 10	Subgraph schema + mappings; deploy to Subgraph Studio	`GraphStateProvider` + onchain reconciliation; daily-spend policy now reads live
Thu 11	Uniswap v4 swap path working from the protected account on Sepolia	`decode/uniswapV4.ts`; **`FEEDBACK.md` written and the Uniswap feedback form submitted**
Fri 12	Verification API (§7.5) deployed at a public HTTPS URL, `/v1/verify` and `/v1/decode` x402-gated through Blocky402; HCS audit topic	`x402-client`; `researcher` agent completes one real paid request; `openapi.yaml` published; **Ledger go/no-go decision**

Gate G3 (end of 12 Sept): one real paid x402 request completes end to end, and the daily-spend decision is driven by live Graph data. These are hard qualification requirements for two $5–6k tracks; nothing after this matters more.

Phase 4 — Agents + UI (13–14 Sept)

Day	Both
Sat 13	LangGraph `trader` agent (real reasoning, not scripted); dashboard split pane + policy rows + proof drawer
Sun 14	`scripts/demo-runner.ts` end to end; the tamper-attempt scene; approval routing; two consecutive green runs

Gate G4 (evening of 14 Sept): the full nine-step demo runs twice in a row without manual intervention. Feature freeze here. Anything unfinished is cut, not rushed.

Phase 5 — Submission (15–16 Sept)

Day	Both
Mon 15	Record the video (main + 2-minute cut). Write README, threat model, formal-verification doc, quickstart. Publish `@agentproof/sdk` to npm. Verify contracts on Etherscan/HashScan. **Bazantic Track A** if the demo is green: gateway + "verify before you swap" recipe + screen recording (≈3 h)
Tue 16	Submit to each track separately with the correct pool selected; walk §18 checklist line by line; buffer for platform issues

Do not write code on 16 September. Bazantic Track B is the only thing that may be attempted on the 16th, and only if every other submission is already filed.

---

16. Risk register

#	Risk	Likelihood	Impact	Mitigation	Trigger → fallback
R1	ERC-7579 hook interface mismatch with the chosen account	High	High	Pin the implementation on Day 1 and read its `HookManager` before writing the module	Not resolved by end of 5 Sept → switch to a standalone `PolicyGuard` executor module and adjust the bypass demo to "the account's executor rejects it"
R2	SMTChecker returns `unknown`	Medium	High	Verify a pure library, small integer widths, two properties only	Not `PROVEN` by 9 Sept → reduce to `MAX_TRANSFER` alone; if still failing, ship Foundry invariant testing + an honest "UNPROVEN" report. **Never claim a proof you do not have**
R3	Blocky402 / Hedera x402 integration friction	Medium	High	Validate the facilitator's `/supported` endpoint on Day 1 of Phase 3; start with the simplest possible gated endpoint	Blocked by 12 Sept → fall back to a direct HTS USDC transfer with an HCS receipt and note honestly that the x402 track requirement was not met
R4	ENSv2 beta instability (contracts explicitly not final)	Medium	Medium	Pin addresses; record them in `deployments/`	Registrar deploy fails → use an existing ENSv2 name with Permissioned Resolver text records + EAC; still qualifies, less impressive
R5	Uniswap v4 Sepolia liquidity is thin or the pool is unusable	Medium	Medium	Check pool state on Day 1 of Phase 3	No usable pool → seed our own v4 pool with mock tokens, or swap on a v3 pool (the bounty accepts v2/v3/v4)
R6	Bundler does not support the chosen 7579 account	Medium	High	Test the bundler on Day 2, not Day 12	Fall back to direct `execute` calls from an EOA owner; the hook still enforces
R7	Scope creep across six sponsors	High	High	§10.0 priorities; Ledger and Bazantic are explicitly droppable	Gate G3 slips → drop P1/P2 sponsors immediately
R8	Live demo fails during judging	Medium	High	Recorded video is the primary artifact; live demo is a bonus	Always have the recording ready to play
R9	Agent behaves unpredictably on camera	Medium	Low	Fix the model, temperature 0, and pin the prompt; the *policy decision* is deterministic even when the agent is not	Re-record; never fake the block with an `if`
R10	Sponsor requirements change mid-event	Medium	Medium	Re-read every prize page on 15 Sept before submitting	Adjust submission text, not architecture

17. Post-hackathon / future work

Put this section in the README — judges reward a credible next step.

- **Multi-asset accounting** with a price oracle, so limits are USD-denominated across any token.
- **Rolling 24-hour window** instead of UTC-day buckets, removing the T6 boundary behavior.
- **Batch execution support** in the hook with per-call decomposition.
- **Policy registry**: policies as ERC-1155 tokens under the ENSv2 registry, transferable and revocable.
- **Certora Prover** specs for the stateful hook, not just the pure library.
- **Additional module types**: a validator that rejects at signature time, so over-limit UserOps never reach the bundler and never cost gas.
- **A policy marketplace** (declared out of scope for the hackathon, credible as a product direction).

18. Submission checklist

18.1 Universal

- [ ] Public GitHub repo, open source, real commit history spread across the event (no single final-day commit dump — 1inch explicitly checks this and other judges notice)
- [ ] README: problem, threat model, architecture diagram, quickstart, what is proven vs fuzzed, known gaps
- [ ] `@agentproof/sdk` published to npm
- [ ] Contracts verified on Etherscan (Sepolia) and HashScan (Hedera testnet)
- [ ] Demo video, main cut ≤ 5 min, plus a 2–4 min cut
- [ ] `deployments/*.json` committed with every live address
- [ ] `proofs/*.json` committed alongside the raw solc log

18.2 Per track

Track	Required artifact	Owner	Done
Hedera — AI & Agentic Payments	Live x402 service on Hedera settled via Blocky402; consuming agent; README covering the payment flow; video ≤ 5 min		☐
The Graph — AI Use Case (**Start Fresh** pool)	Live Graph data via Studio API key; the data drives a decision; README/SKILL.md; video 2–4 min; **correct pool selected**		☐
ENS — Best Use of ENSv2	Sepolia ENSv2; registry/resolver/EAC central, not cosmetic; no hard-coded values; video and/or live demo link		☐
Uniswap — Best Stack Contribution	`FEEDBACK.md` in repo + **feedback form submitted with the link**; README points at exact contracts and lines		☐
Ledger — AI Agents × Ledger	Built on the Ledger Agent Stack; device confirmation before an irreversible action		☐
Bazantic — Best Recipe	Gateway + recipe chaining two sponsor services; screen recording; **Bazantic username in the submission**		☐
Bazantic — Agentify a new API	A genuinely new API added to Bazantic + recipe + recording		☐

18.3 Things that lose prizes on technicalities

1. Forgetting Uniswap's `FEEDBACK.md` and the feedback form.
2. Selecting the Continuity pool when you are net-new (or vice versa).
3. Demoing The Graph from a fixture instead of live data.
4. A Hedera submission where nothing is actually *hosted* behind x402.
5. An ENS integration that only reads a name.
6. A video over the length limit.
7. Omitting the Bazantic username, which is how they attribute your recipe.

---

19. Self-assessment against ETHOnline 2026

19.1 Projected scoring, if executed to Gate G4

Dimension	Score	Note
Concept and narrative	9/10	"Trust the agent to decide, not to enforce" is a memorable line attached to a real, current problem
Architectural coherence	8/10	One product, one policy engine, two enforcement layers with clearly separated trust assumptions
Security soundness	8/10	Measured outflow rather than declared amounts; explicit threat model; stated non-defenses
Formal-verification realism	8/10	A real proof on a deliberately small surface, plus an honest counterexample and an honest limitations section
Sponsor-requirement fit	8/10	Every integration is load-bearing and mapped line-by-line to published qualification requirements
Executability in 12 days	7/10	Tight. The gates in §15 are what protect it
Overall	**8/10**	

19.2 Expected placement by track

Track	Pool	Odds of placing	Reasoning
ENS — Best Use of ENSv2	$4,500	**Good**	Agents-as-namespaces with EAC-delegated roles is exactly the bonus they named, and few teams will deploy their own ENSv2 subname registry
Hedera — AI & Agentic Payments	$6,000	**Good, conditional**	Only if we genuinely host the x402 service. The field will be crowded with payment demos; "the thing being sold is a safety verification, and the payment is itself policy-checked" is the differentiator
The Graph — AI Use Case	$5,000	**Moderate**	Load-bearing use is clear, but the track attracts many strong agent submissions. Graph-vs-onchain reconciliation is our edge
Uniswap — Stack Contribution	$3,000	**Moderate**	Up to three teams win; a reusable v4 calldata decoder for agent safety is a genuine ecosystem contribution
Ledger — AI Agents	$3,500	**Good if attempted**	The track description reads as if written for this project. Likely a smaller field
Bazantic — Recipe	$1,000	**Moderate**	Small field, low effort once the API exists, and our recipe is a real workflow rather than a contrivance

19.3 The three things that most improve the odds

1. **Shot 7 of the demo** — bypassing the SDK and watching the account revert. It is the only moment that proves the thesis rather than describing it.
2. **The counterexample** — showing what the solver says when the property is broken proves the verification is real. Two green checkmarks prove nothing.
3. **Hosting the x402 service rather than only consuming it** — it unlocks the largest single pool on offer, and the Verification API it produces is what makes Bazantic nearly free.

19.4 The single biggest risk

Attempting six sponsors and finishing none of them well. The gates in §15 exist for exactly that reason. **Four complete integrations beat six partial ones, every time.**